Runtime Boundaries

Purpose

Every Runtime within Shio Runtime owns a clearly defined architectural boundary.

A boundary defines where one Runtime's responsibility ends and another Runtime's responsibility begins.

Clear boundaries prevent responsibility overlap, reduce coupling and simplify future evolution.

Whenever uncertainty exists about where new functionality belongs, the Runtime Boundaries should be consulted before implementation begins.

Why Boundaries Matter

Architectural quality depends more on clear boundaries than on internal implementation.

Poorly defined boundaries gradually create:

- duplicated logic;
- circular dependencies;
- hidden side effects;
- difficult maintenance.

Strong boundaries allow Runtime systems to evolve independently while preserving overall architectural consistency.

Responsibility Transfer

Information flows through the Runtime as responsibilities are transferred from one subsystem to another.

Each Runtime completes its work before handing responsibility to the next architectural layer.

No Runtime continues operating outside its own responsibility.

Speech Runtime Boundary

Speech Runtime begins with communication.

Its responsibility includes:

- receiving speech;
- recognizing speech;
- understanding intent;
- generating responses;
- synthesizing speech.

Speech Runtime ends when communication has been transformed into either:

- Character State updates;
- Events;
- Action intentions.

Speech Runtime never executes external actions.

Decision Pipeline Boundary

Decision Pipeline evaluates intent.

Its responsibility is determining:

- whether input should be ignored;
- whether Shio should respond;
- whether an action is requested;
- whether reasoning is required.

Decision Pipeline ends once a decision has been made.

Execution belongs elsewhere.

Action Runtime Boundary

Action Runtime begins when an external side effect has been requested.

Its responsibility includes:

- validation;
- authorization;
- execution;
- adapter selection;
- result generation.

Action Runtime ends when execution completes.

It never determines emotional reactions.

Experience Layer Boundary

Experience Layer begins after something has happened.

Its responsibility is interpreting completed events.

Examples include:

- success;
- failure;
- interruption;
- unexpected outcomes.

Experience Layer decides how Shio emotionally reacts.

It never performs execution.

Avatar Runtime Boundary

Avatar Runtime begins when the Character State changes.

Its responsibility is visual representation.

Avatar Runtime transforms state into:

- body movement;
- facial expressions;
- gestures;
- breathing;
- blinking;
- lipsync.

Avatar Runtime never generates Character State.

It represents it.

Memory Layer Boundary

Memory Layer begins when continuity may need to be preserved.

Its responsibility includes:

- evaluating significance;
- storing memories;
- retrieving memories;
- forgetting insignificant information.

Memory Layer never generates dialogue.

It provides context.

External Adapter Boundary

Adapters begin where Runtime responsibilities end.

They translate Runtime concepts into technology-specific operations.

Adapters never contain Runtime logic.

Adapters never define behavior.

Adapters exist solely to bridge architecture and implementation.

Example — Complete Responsibility Flow

Consider the following interaction:

The user says:

> "Shio, ban this spammer."

Responsibility flows through the architecture as follows:

```text id="rb01" Speech Runtime

↓

Decision Pipeline

↓

Action Runtime

↓

Platform Adapter

↓

External Service

↓
```

Action Result

↓

Experience Layer

↓

Character State

↓

Avatar Runtime ```

Each Runtime performs exactly one responsibility.

No responsibility overlaps another.

Responsibility Ownership

Whenever new functionality is introduced, it should belong to exactly one Runtime.

If multiple Runtime systems appear equally responsible, the architecture should be reconsidered.

Responsibility duplication is considered an architectural smell.

Boundary Questions

Before implementing a new feature, the following questions should always be answered:

1. Which Runtime owns this responsibility?
2. Where does this Runtime begin?
3. Where does this Runtime end?
4. What information crosses the boundary?
5. Is Character State sufficient?
6. Is an Event required?
7. Does this feature require a new Runtime?

These questions should be answered before writing implementation code.

Architectural Guarantees

Clearly defined Runtime boundaries guarantee:

- single ownership;
- predictable responsibilities;
- independent Runtime evolution;
- easier debugging;
- reduced coupling;
- simpler code reviews.

Well-defined boundaries are one of the strongest indicators of a healthy Runtime architecture.

Final Rule

Whenever there is uncertainty about where new code belongs, do not ask:

> "Which file should this go into?"

Instead ask:

> **"Which Runtime owns this responsibility?"**

The answer to that question should determine the implementation.

This principle should guide every future architectural decision within Shio Runtime.